

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	S.GAYATHERI	Reg.No.:	192521306
Department:	B.TECH Information Technology	Date:	20/08/26

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

Problem

A smartphone experiences lag when switching between applications. This occurs because the CPU frequently needs data that may not be present in the fastest cache levels, causing accesses to slower memory.

Role of Memory Levels

Memory	Role	Approximate Access
L1 Cache	Stores most frequently used instructions/data	~1 ns
L2 Cache	Stores recently used data not in L1	~4 ns
L3 Cache	Larger shared cache for CPU cores	~12 ns

DRAM	Stores active applications and OS data	~80 ns
------	--	--------

Existing Hierarchy

CPU → L1 → L2 → L3 → DRAM → Storage

When an application is switched, frequently accessed data should ideally remain in L1/L2/L3. If it is unavailable, the CPU must access DRAM, increasing latency.

Calculation

Assume:

- L1 hit time = 1 ns
- L1 miss rate = 5%
- L2 hit time = 4 ns
- L2 miss rate = 10%
- L3 hit time = 12 ns
- L3 miss rate = 5%
- DRAM = 80 ns

Thus, an efficient cache hierarchy significantly reduces the average memory access time compared with directly accessing DRAM.

Proposed Redesign

CPU → Larger L1/L2 → Shared L3 → High-bandwidth LPDDR5X DRAM → UFS Storage

Improvements:

- Optimize L1 and L2 cache utilization.
- Increase the effective L3 cache capacity.
- Use high-bandwidth LPDDR5X DRAM.
- Use cache prefetching for application data.
- Keep frequently used application states in memory.

Result

The redesigned hierarchy reduces DRAM accesses, decreases app-switching latency, improves CPU throughput, and provides a smoother smartphone experience.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

Problem

A cloud database server experiences high query latency. The problem may result from both **cache misses** and **virtual-memory paging**.

Cache Hierarchy vs Virtual Memory

Feature	Cache Hierarchy	Virtual Memory Paging
Purpose	Reduce CPU memory latency	Extend usable memory
Managed mainly by	Hardware/CPU	OS
Fastest storage	L1/L2/L3	DRAM
Slow path	DRAM	SSD
Main problem	Cache miss	Page fault
Effect	Small latency increase	Very large latency increase

Cache Hierarchy

CPU → L1 → L2 → L3 → DRAM

A database query repeatedly accessing the same data benefits from CPU caches and the database's own buffer/cache.

Virtual Memory

If DRAM becomes full, inactive pages may be moved to SSD.

DRAM → Page Out → NVMe SSD

When required again:

NVMe SSD → Page In → DRAM

This creates a **page fault**, which is much slower than a cache miss.

Example Calculation

Suppose:

- Normal DRAM access = 100 ns
- Page fault = 100 μ s
- Page-fault rate = 0.01%

Since:

Even a small page-fault rate can significantly increase average access time.

Recommended Design

CPU → L1 → L2 → L3 → Large DRAM → NVMe SSD

Improvements:

- Increase DRAM capacity.
- Use high-bandwidth DDR5 memory.
- Use large database buffer pools.
- Reduce unnecessary paging.
- Use fast NVMe SSD for unavoidable paging.
- Optimize database queries and indexes.
- Use memory-aware workload allocation in cloud servers.

Result

Increasing DRAM and reducing page faults provides a much greater improvement than simply increasing cache size when the main bottleneck is virtual-memory paging.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Problem

An autonomous vehicle continuously processes camera, LiDAR, radar, GPS, and other sensor data. Processing must happen with extremely low latency and high bandwidth.

Memory Comparison

Memory	Speed	Volatility	Main Use
SRAM	Very high	Volatile	CPU/AI accelerator cache
DRAM	High	Volatile	Large sensor datasets
MRAM	High	Non-volatile	Persistent data/configuration

Requirements

The system requires:

- Very low latency for real-time decisions.
- High bandwidth for sensor streams.
- Sufficient capacity for large datasets.
- Low power consumption.

Proposed Hierarchy

CPU/AI Accelerator → SRAM → L2/L3 Cache → High-Bandwidth DRAM → MRAM → Flash Storage

Working

SRAM

Stores frequently accessed sensor information and AI intermediate calculations.

DRAM

Stores large camera frames, LiDAR point clouds, radar data, and active AI models.

MRAM

Stores persistent configuration, calibration information, and selected frequently required data without losing information when power is removed.

Example Bandwidth Calculation

Assume four sensors each generate 2 GB/s:

If additional processing requires 50% overhead:

Therefore, the DRAM subsystem should provide **at least 12 GB/s**, preferably with additional headroom.

Recommended Design

- Use large on-chip SRAM for immediate processing.
- Use high-bandwidth DRAM for sensor datasets.
- Use MRAM for non-volatile information.
- Use DMA to transfer sensor data directly between memory and accelerators.
- Use parallel memory channels to increase bandwidth.

Result

The proposed hierarchy provides **low latency through SRAM, high bandwidth through DRAM,** and **non-volatile storage through MRAM**, making it suitable for real-time autonomous driving.

4. AI Inference Edge Device
- An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

Problem

An edge AI device needs to load AI models quickly while consuming little power. Large models cannot be stored entirely in expensive high-speed SRAM.

Memory Comparison

Memory	Speed	Capacity	Power	Main Purpose
SRAM	Very high	Very small	Higher	AI cache/activation
DRAM	High	Large	Medium	Active model
RRAM	High	Medium	Low	Non-volatile model storage

NAND Flash	Lower	Very large	Low	Permanent storage
------------	-------	------------	-----	-------------------

Existing Problem

If every AI model is loaded directly from NAND Flash:

NAND → DRAM → AI Accelerator

This increases model-loading time.

Proposed Hierarchy

AI Accelerator → SRAM → DRAM → RRAM → NAND Flash

Working

SRAM

Stores frequently used weights, activations, and intermediate calculations.

DRAM

Stores the currently active AI model and larger working datasets.

RRAM

Stores frequently used AI models in non-volatile form and can provide faster access than conventional NAND storage.

NAND Flash

Stores large collections of models and permanent data.

Example

Suppose:

- Model size = 2 GB
- NAND transfer rate = 500 MB/s

Approximate loading time:

If a faster memory layer can provide 2 GB/s:

This demonstrates how moving frequently used models to a faster memory layer can reduce loading time.

Recommended Design

- SRAM:** critical model weights and activations
- DRAM:** active model and runtime data
- RRAM:** frequently used models
- NAND:** large permanent model library

Result

The proposed hierarchy reduces model-loading time, reduces unnecessary NAND accesses, improves inference performance, and lowers energy consumption.

- 5. High-Performance Gaming Console
 - A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Problem

A gaming console experiences frame drops when loading large game scenes. Modern games require large textures, geometry, shaders, and other assets to be transferred rapidly to the CPU/GPU.

L3 Cache vs DRAM

Feature	L3 Cache	DRAM
Latency	Very low	Higher
Capacity	Smaller	Much larger
Bandwidth	High	Very high
Main purpose	Frequently reused data	Large game datasets
Best for	CPU/GPU working data	Textures, assets, game scenes

L3 cache is useful for frequently reused data, but it cannot store an entire large game scene.

DRAM provides the capacity and bandwidth needed by the GPU.

Existing Hierarchy

CPU/GPU → L1/L2 → L3 → DRAM → SSD

When a large scene is loaded:

SSD → DRAM → GPU

If this transfer is slow, frames may be delayed and frame drops can occur.

Example Bandwidth Requirement

Suppose a scene requires 8 GB of assets to be transferred in 2 seconds:

Therefore, the memory subsystem must provide more than **4 GB/s**, preferably with additional bandwidth for processing.

Proposed Design

CPU/GPU → L1/L2 → Large/Optimized L3 → High-Bandwidth GDDR6/GDDR6X DRAM → NVMe SSD

Improvements:

- Optimize L3 cache for frequently reused game data.
- Use high-bandwidth DRAM for textures and large assets.
- Use SSD asset streaming.
- Implement predictive prefetching.
- Compress game assets to reduce transfer volume.
- Transfer data asynchronously before the scene is required.

Result

L3 cache reduces latency for frequently accessed data, while high-bandwidth DRAM handles large textures and game assets. Faster SSD streaming and prefetching reduce loading delays and help maintain stable frame rates.

Code of Conduct:

*I S.Gayatheri (192521306) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
S.GAYATHERI

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis